

Тестовое задание для Frontend разработчика

Важно: Ссылку на выполненное тестовое необходимо указать в резюме в блок «О себе».

Срок выполнения: 2 дня

Формат сдачи: ссылка на Github

Стек: React.js

Бизнес кейс: AI-агент проверки целевого использования льготных кредитов

Проект: AI-агент для автоматизации проверки документов, планируемых к заявлению в банк для оплаты за счёт средств льготного кредита.

Проблема: Специалисты финансового блока тратят время на ручную проверку документов в разрозненных системах. Отсутствие единого интерфейса приводит к ошибкам выбора программы проверки и затрудняет анализ причин отказа. Проект должен предоставить интуитивный и надежный веб-интерфейс для централизованного управления проверками и наглядного отображения результатов.

Как процесс работает сейчас (AS IS)

Специалист скачивает шаблоны реестров, вручную заполняет таблицы в Excel, сверяет данные с бумажными копиями документов. Для поиска причины отказа нужно открывать десятки файлов. Нет единого окна для контроля статуса всех заявок.

Как процесс должен работать (TO BE)

- Специалист загружает пакет документов через веб-интерфейс и выбирает льготную программу из справочника.
- Интерфейс отображает статус проверки в реальном времени ("в очереди", "проверяется", "готово").
- По завершении проверки система показывает итоговый статус ("можно заявлять", "нельзя заявлять", "требуется ручная проверка").
- При статусе "нельзя заявлять" или "требуется ручная проверка" интерфейс подсвечивает конкретные причины: отсутствующий документ, нарушение срока, расхождение сумм.
- Для авансовых платежей интерфейс позволяет загрузить закрывающие документы и запустить пост-контроль с визуальным сопоставлением позиций (что оплачено vs что поставлено).
- Руководитель видит дашборд со списком спорных кейсов и кратким отчетом по каждой заявке.

Ключевые сценарии использования (User Stories)

- Как педагог я хочу, чтобы можно было заполнять полные данные о нём.
- Как педагог я хочу, чтобы методики работы с детьми были оформлены в читаемый каталог, в котором можно находить нужные активности и упражнения.

1. Тестовое задание

Ссылка на API: https://gitverse.ru/semao0/mock_api_credit

Задача: Реализовать веб-приложение для загрузки документов и отображения результатов проверки. Приложение работает с готовым mock API (инструкция по запуску ниже).

- 1) Реализовать форму загрузки: drag & drop зона для файлов (PDF, DOCX, JPG, PNG) и выбор льготной программы («Федеральная» / «Областная»). Кнопка запуска проверки активна только при наличии файлов и выбранной программы.
- 2) Отображать состояние загрузки во время обработки запроса (сервер отвечает с задержкой ~5 сек).
- 3) Показывать карточку результата проверки: цветовой статус (зелёный / красный / жёлтый), краткое объяснение, список выявленных проблем с разбивкой на ошибки и предупреждения, извлечённые данные (контрагент, сумма, дата, предмет оплаты).

- 4) Реализовать кнопку «Скачать отчёт» — выгрузка результата проверки в JSON-файл.
- 5) Сделать экран истории проверок: таблица с датой, программой, количеством документов и статусом. Фильтрация по статусу. Клик по строке открывает детали проверки.
- 6) Корректно обрабатывать состояния: загрузка, ошибка сети, пустая история.

Требования к интерфейсу: адаптивность, семантическая разметка, соблюдение базовых принципов доступности (a11y), аккуратный и читаемый UI. Допускается использование UI-библиотек (MUI, Ant Design, shadcn/ui) или самостоятельная стилизация. Специалист финансового блока должен с первого взгляда понять статус проверки, найти причину отказа и не допустить ошибку при выборе программы.

Оценивается: иерархия информации, читаемость статусов, поведение в граничных состояниях (ошибка, пустой список, долгая загрузка). Адаптивная вёрстка (десктоп от 1280px).

Технические требования:

Стек: React.js.

Приложение должно запускаться через docker compose up — предоставить Dockerfile и docker-compose.yml.

Репозиторий должен содержать README.md с инструкцией запуска (не более 3 команд).

Mock API: подними локально перед началом работы.

```
# нужен Docker

docker compose up --build

# или локально

pip install -r requirements.txt && uvicorn app.main:app --reload
```

Документация эндпоинтов: <http://localhost:8000/docs>

Результат проверки зависит от имён загруженных файлов — называй их с ключевыми словами (договор, счёт, спецификация, акт), чтобы получать разные статусы. Сервер имитирует задержку ~5 сек.

2. Мотивация участия в проекте:

1. Почему тебе интересен этот проект?
2. Как ты видишь свою роль в команде, которая создаёт продукт?
3. Сколько времени в неделю ты готов(а) уделять проекту и в течение какого периода?